

Patuakhali Science and Technology University
Faculty of Computer Science and Engineering

CIT 324 :: Simulation and Modeling Sessional

Project Report



CloudCalc

FLOW

Project Title : CloudCalc Flow

Submission Date : 12 July 2026

Submitted from,	Submitted to,
Md. Sharafat Karim ID : 2102024, Reg : 10151, Semester : 6 (Level-3, Semester-2)	1. Md Mahbubur Rahman Associate Professor, Department of Computer Science and Information Technology, Patuakhali Science and Technology University. 2. Dr. Md Abdul Masud Professor, Department of Computer Science and Information Technology, Patuakhali Science and Technology University.

Contents

1. Introduction	3
2. Objectives	3
3. Problem Statement	3
4. Literature Review	3
5. Related Commercial Projects	4
6. Scope	4
6.1. Job Market Analysis	4
7. Methodology	5
7.1. Technology Stack	5
7.2. Design Principles	6
8. Simulation Model	6
9. Feature Comparison	7
10. Visual Models	7
10.1. Flow Chart	7
10.2. Data Flow	7
10.3. Connection Rules	8
10.4. Timeline (Gantt Chart)	8
11. User Interface	9
12. Security and Privacy	10
13. Future Plans	10
14. Result	10
15. Conclusion	10
16. References	11



CloudCalc Flow

Visual Cloud Architecture Simulator and Cost Estimator

1. Introduction

CloudCalc Flow is a client-side, visual cloud architecture simulator and cost estimator. It lets a user arrange infrastructure components like compute servers, load balancers, API gateways, databases, object storage, in-memory cache, a CDN and a message queue to simulate a variable load and estimate costs.

2. Objectives

1. To implement a transparent, real-time simulation of RAM utilisation, response time and throughput as a function of the simulated concurrent-user load.
2. To estimate the total monthly cost alongside detecting overloaded servers.
3. To provide a user friendly interface to interact with the simulator including autosaving, multiple theming and responsive design.
4. To build a node-based canvas for composing cloud architectures through drag-and-drop support.

3. Problem Statement

Planning a cloud deployment forces two questions to be answered at once:

1. what will it look like?
2. what will it cost and sustain?

Existing tools tend to answer only one. Native cloud pricing calculators (AWS, Google Cloud, Azure) produce accurate bills but present a flat list of line items. Diagramming tools (draw.io, Lucidchart) capture topology beautifully but know nothing about cost or performance. Visual cost tools such as Cloudcraft bridge some of this gap but are provider-locked, require an account, and keep the model opaque.

To address this, this project implements a fully client-side, open-source, provider-agnostic simulator that combines a node-based architecture canvas with a transparent, testable model of RAM utilisation, response time and throughput. It produces a live cost estimate and flags overloaded servers with upgrade/scale/reduce suggestions.

4. Literature Review

The relationship between server utilisation and responsiveness is a classical result of queueing theory. Kleinrock [1] shows that for an M/M/1-style service, mean response time rises sharply and non-linearly as utilisation approaches 100%, which motivates CloudCalc Flow's response-time penalty that grows with RAM utilisation and its throughput degradation term near

saturation. Little's Law [2] — that the mean number in a system equals arrival rate times mean time in system — underpins the intuition that throughput and concurrency cannot both grow without bound on fixed resources.

For capacity planning specifically, Menascé and Almeida [3] formalise metrics-driven models that map workload intensity (here, concurrent users) onto resource demand and service level. At the platform level, Armbrust et al. [4] frame elasticity and pay-as-you-go economics as the defining properties of cloud computing.

Finally, the practice of treating cloud spend as an engineering concern is codified by the FinOps Foundation [5], whose framework argues that engineers should see the cost consequences of architectural choices in real time. CloudCalc Flow is a small, educational embodiment of that principle.

5. Related Commercial Projects

1. **AWS Pricing Calculator** [6] gives authoritative, up-to-date pricing for real AWS services, but produces a flat itemised estimate with no architecture diagram and no load or performance simulation.
2. **Google Cloud** [7] and **Azure** [8] calculators are equivalent in spirit — accurate, provider-specific, list-based — and likewise omit topology and capacity behaviour.
3. **Cloudcraft** [9] is the closest visual analogue: it renders live AWS/Azure diagrams with attached cost. However, it is provider-locked, requires an account, is not open source, and does not simulate user-driven load against per-node capacity.
4. **React Flow** / **xyflow** [10] is not a competitor but the enabling library — a node-based editor toolkit — on which CloudCalc Flow's canvas, custom nodes, edges and connection validation are built.

6. Scope

CloudCalc Flow is delivered as a modern web application built with Next.js and React Flow. It is entirely client-side without any server, database or authentication, and no data ever leaves the browser. The application ships nine component types with a hardcoded but clearly-documented cost and capacity catalogue, a strict connection-rule engine, a pure simulation module, undo/redo history, localStorage persistence, JSON export/import and a markdown summary.

6.1. Job Market Analysis

The job market for full stack web developers in Bangladesh is growing rapidly, with significant demand for Next.js and modern JavaScript expertise. According to major job portals:

Job Portal	Link	Stack
BD Jobs	1. https://bdjobs.com/jobs/details/1449039?ln=1 2. https://bdjobs.com/jobs/details/1449038?ln=1	React, NodeJS, mongoDB, mongoose, JavaScript, Github, Git

	3. https://bdjobs.com/jobs/details/1452423?ln=1	
BD Tech Jobs [11] & Techn-Talents [12]	1. https://authlab.io/wpjb-jobs/senior-software-engineer/ 2. https://riseuplabs.com/jobs/full-stack-developer-python-django/	Vue.js, React
eJobs Bangladesh [13] & JobMatchingBD [14]	1. https://www.ejobs.com.bd/jobs/san-francisco-ca-senior-full-stack-developer 2. https://jobmatchingbd.com/job/engineering-jobs-14/ 3. https://jobmatchingbd.com/job/it-jobs-in-bangladesh-second-source/	Node.js, React/ Vue, PostgreSQL or, MongoDB
Arc.dev [15] & Wellfound [16]	1. https://wellfound.com/jobs/2993781-senior-react-and-nextjs-web-developer 2. https://wellfound.com/jobs/2927551-senior-full-stack-developer	React, npm, TypeScript, Next.js and Git
Careerjet Bangladesh [17]	1. https://www.careerjet.com.bd/jobad/bdbe02ffc4686d4de58c19bc2c53c6f999 2. https://www.careerjet.com.bd/jobad/bda7f026ae3ccb8bd6ab7d9027f8d5c26f	React, NodeJS, mongoDB, mongoose, JavaScript, Github, Git, Tailwind CSS

Table 1: Job Market Opportunities for Next.js & Full-stack Developers in Bangladesh

The demand for full stack developers with Next.js expertise is becoming the standard for modern web development in Bangladesh, making this project a timely project that demonstrates bleeding edge skill technology.

7. Methodology

7.1. Technology Stack

Development followed an iterative, agile approach. The stack was selected for performance, type-safety and a fully client-side deployment.

Component	Technology
Frontend Framework	Next.js 16 (React 19, App Router, Turbopack)
Language	TypeScript (strict)
Canvas / Graph	React Flow v12 (@xyflow/react)
State Management	Zustand 5 with immer + persist middleware
Styling	Tailwind CSS v4
Icons	Lucide React
Persistence	Browser localStorage (autosave) + JSON export/import
Testing	Vitest (pure simulation unit tests)
Diagrams	Mermaid; report typeset in Typst

Table 2: Technology stack for CloudCalc Flow

7.2. Design Principles

1. All cost and performance math lives in dependency-free functions (`lib/simulation.ts`) so it can be unit-tested and reused by both the nodes and the metrics panel.
2. A Zustand store holds nodes, edges and the user-count; every view derives from it, and history snapshots wrap each mutation for undo/redo.
3. No backend, no login; the whole architecture autosaves to `localStorage` and can be exported as a portable JSON file.
4. Type checking and linting are enforced at build time, and the production build is verified to compile cleanly.

8. Simulation Model

Here N_{servers} is the number of servers sharing traffic. This mirror the qualitative behaviour of a saturating queue [1]. Metrics are: total monthly cost, total RAM, traffic-weighted average response time, peak throughput, estimated bandwidth, and the worst node status.

Quantity	Definition
Base RAM	OS overhead + framework idle
Traffic RAM	$(\text{users} / N_{\text{servers}}) / 100 \times \text{stackPer100}$
Utilisation	$(\text{Base} + \text{Traffic}) / \text{TierRAM} \times 100\%$
Response time	$\text{TierBaseline} + \text{Util} / 100 \times 50\text{ms}$
Throughput	$\text{TierMax} \times (1 - \text{Util} / 150)$

Table 3: Per-server simulation formulas

The catalogue is summarised below.

Tier	RAM	vCPU	Cost/mo	Max req/s
Micro	1 GB	2	\$8	100
Small	2 GB	2	\$16	500
Medium	4 GB	4	\$32	2000
Large	8 GB	8	\$64	5000

Table 4: Compute server tiers

Component	Cost/mo	Stack (RAM / 100 users)	idle
Load Balancer	\$20	Next.js — 150 MB	120 MB
API Gateway	\$15	Express.js — 80 MB	60 MB
SQL Database	\$8–10 +/GB	Flask — 50 MB	40 MB
NoSQL Database	\$12–15 +/GB	Go — 20 MB	15 MB
S3 Storage	\$5 + \$0.10/GB	OS: Ubuntu 400 MB	
Redis Cache	\$15	OS: Alpine 50 MB	
CDN	\$20 + \$0.08/GB	OS: Arch 100 MB	
Message Queue	\$25		

Table 5: Component costs and memory footprints (documented defaults)

9. Feature Comparison

Table 6 contrasts CloudCalc Flow with widely used cloud pricing and diagramming tools.

Tool	Visual canvas	Live cost	Traffic sim	Perf metrics	Client-side	No login	Export	FOSS
CloudCalc Flow	✓	✓	✓	✓	✓	✓	✓	✓
AWS Pricing Calc.	—	✓	—	—	—	✓	✓	—
GCP Calculator	—	✓	—	—	—	✓	✓	—
Azure Calculator	—	✓	—	—	—	✓	✓	—
Cloudcraft	✓	✓	—	—	—	—	✓	—
draw.io	✓	—	—	—	✓	✓	✓	✓

Table 6: Feature comparison of CloudCalc Flow with existing tools

10. Visual Models

10.1. Flow Chart

This shows the flow of control from user action to the final rendered metrics. The store is updated via an immer mutation, which triggers a recompute of all derived values and a persistence to localStorage.

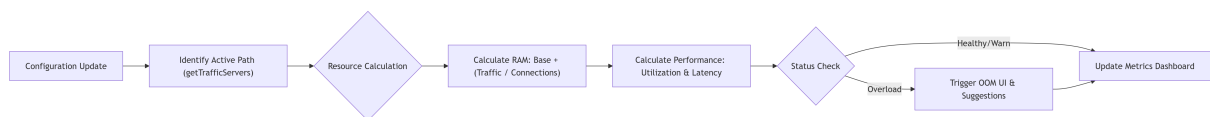


Figure 1: The calculation loop from a slider/config change to per-node status and global metrics

10.2. Data Flow

This shows the flow of data from user action to the final rendered metrics. The store is updated via an immer mutation, which triggers a recompute of all derived values and a persistence to localStorage.

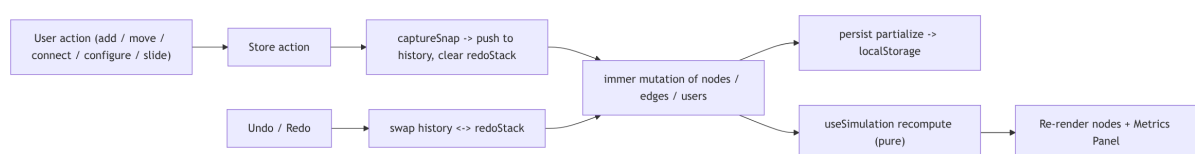


Figure 2: Action to render: history snapshot, immer mutation, persistence and pure recompute

10.3. Connection Rules

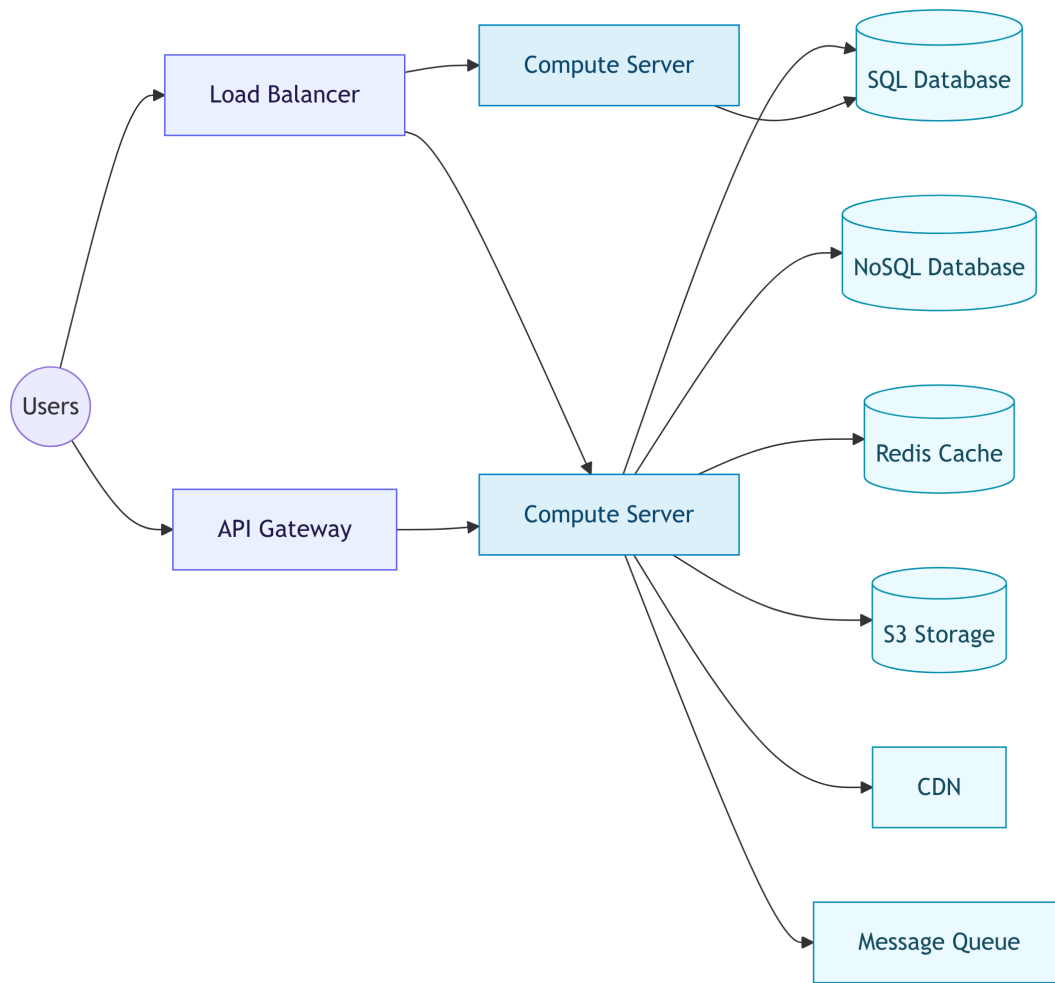


Figure 3: Enforced topology — entry points route to servers; servers write to data, cache, edge and queue sinks

10.4. Timeline (Gantt Chart)

Task	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6
Data models + simulation math + tests	✓					
Zustand store + history + connection rules		✓				
Canvas, custom nodes and edges		✓	✓			
Palette, metrics panel, config modal			✓	✓		
Warnings, suggestions, export, shortcuts				✓		
Extended components (CDN, gateway, queue, NoSQL)					✓	
Dark mode, responsiveness, verification					✓	✓

Table 7: Development timeline of CloudCalc Flow

11. User Interface

This UI is designed to be intuitive and responsive, with components on the left, a central canvas for arranging nodes, and a metrics panel on the right.

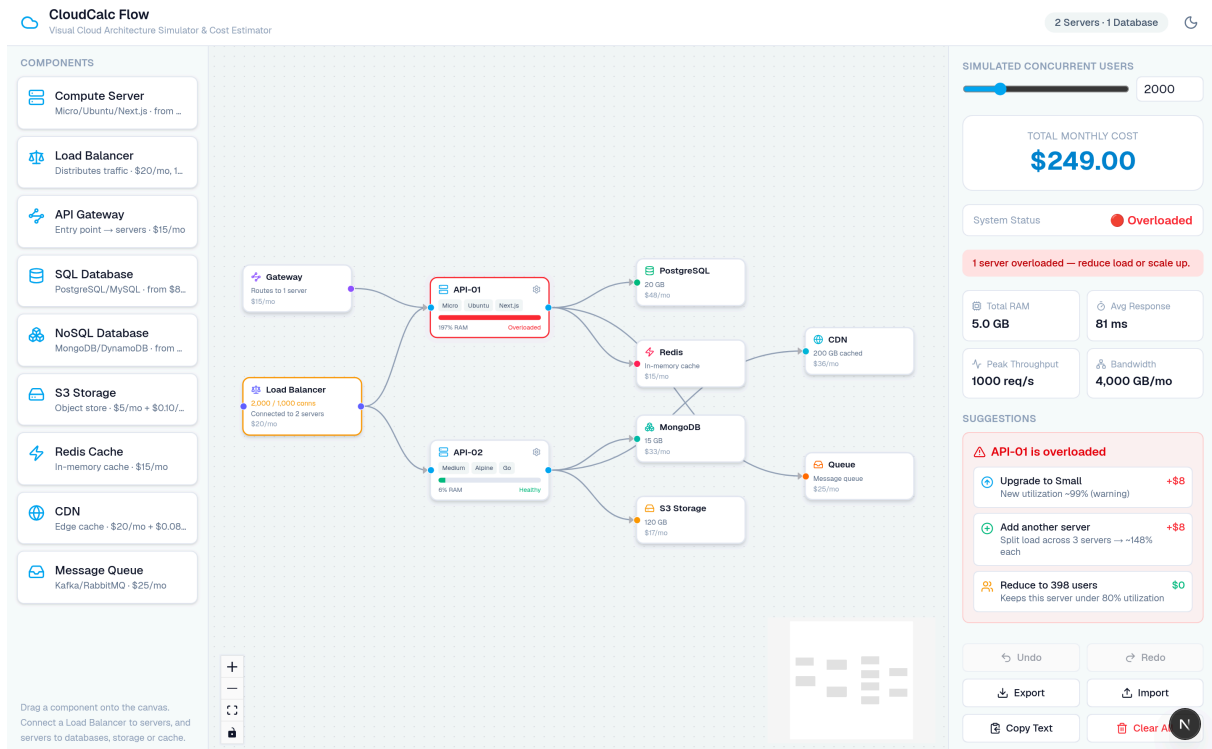


Figure 4: Home view of the simulation

Nodes are also configurable in order to do vertical scaling.

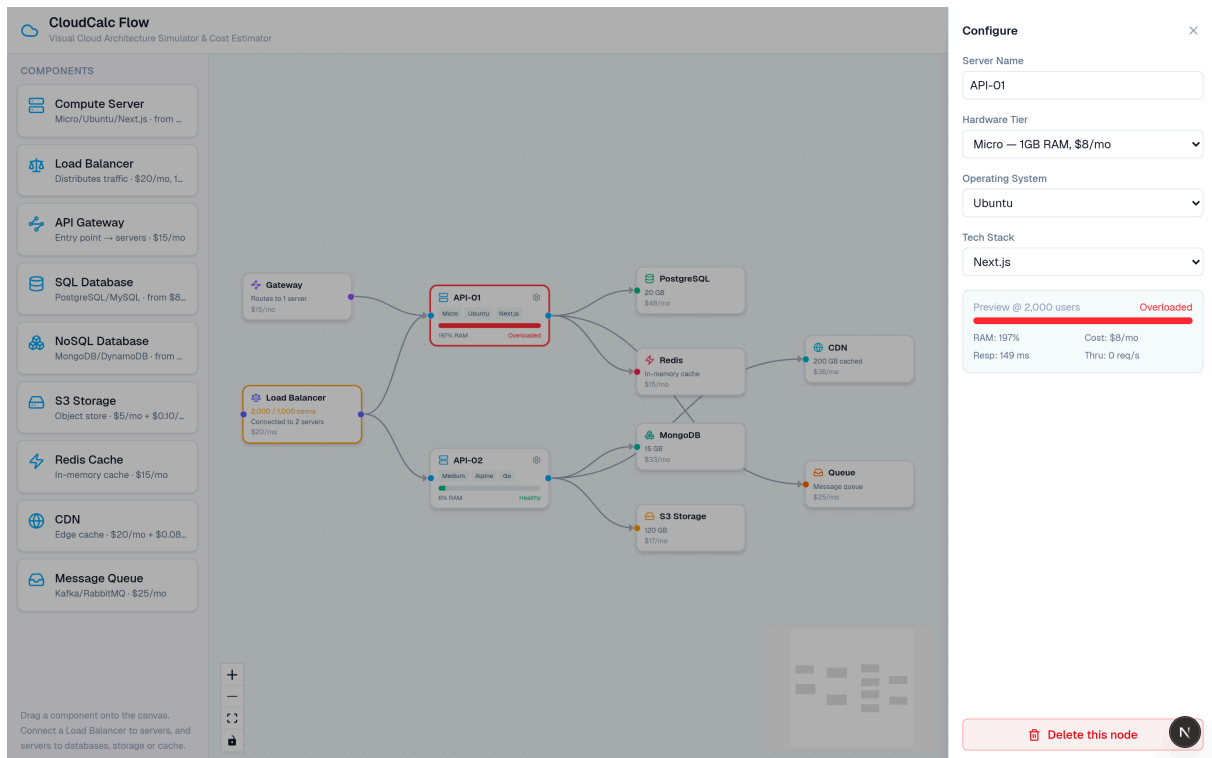


Figure 5: Component configuration modal for a selected node

12. Security and Privacy

CloudCalc Flow is private by construction. It has no backend, no authentication and no network calls that carry user data:

1. The entire architecture lives in browser memory and is autosaved only to the local device's `localStorage`.
2. Export produces a plain JSON file the user chooses to save
3. Import reads a local file. Because there is no server, there is no attack surface for data exfiltration, no account to breach and no telemetry.

This aligns with a privacy-first, offline-capable design and sidesteps the account and data-residency concerns of hosted cost tools.

13. Future Plans

1. Provider-specific pricing profiles (AWS/GCP/Azure) selectable at runtime, replacing the single documented catalogue.
2. Preset architectures (“scalable web app”, “microservices”) and side-by-side cost comparison of two designs.
3. Richer traffic modelling — per-edge bandwidth, autoscaling policies and multi-region deployment.
4. Infrastructure-as-Code export (Terraform / Docker Compose) generated from the canvas.

14. Result

1. A fully client-side simulator was delivered with nine component types, drag-and-drop composition and enforced, type-safe connection rules.
2. A transparent per-server model computes RAM utilisation, response time and throughput live, aggregates a monthly cost, and flags overloaded nodes with cost-annotated upgrade/scale/reduce suggestions — verified end-to-end (see Figure 4).
3. Undo/redo, `localStorage` autosave, JSON export/import, copy-as-text, keyboard shortcuts, light/dark theming and responsive layout were all implemented.
4. The simulation math is covered by passing unit tests, and the production build compiles cleanly.

15. Conclusion

CloudCalc Flow demonstrates that architecture visualisation and cost/performance reasoning can be unified in a single, instant, no-login web tool. By pairing a node-based canvas with a small, transparent and unit-tested simulation model, it turns an otherwise static diagram into an interactive capacity-and-budget sandbox. The design keeps all data on the device, exposes every constant for teaching, and remains extensible toward real provider pricing and infrastructure-as-code export. It is both a usable planning aid and to understand the trade-offs of cloud deployment.

16. References

- [1] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [2] J. D. C. Little, “A Proof for the Queuing Formula: $L = \lambda W$ ”, *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961, doi: 10.1287/opre.9.3.383.
- [3] D. A. Menascé and V. A. F. Almeida, *Capacity Planning for Web Services: Metrics, Models, and Methods*. Prentice Hall, 2002.
- [4] M. Armbrust *et al.*, “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010, doi: 10.1145/1721654.1721672.
- [5] FinOps Foundation, “What is FinOps? Cloud Financial Management Framework.” [Online]. Available: <https://www.finops.org/introduction/what-is-finops/>
- [6] Amazon Web Services, “AWS Pricing Calculator.” [Online]. Available: <https://calculator.aws/>
- [7] Google Cloud, “Google Cloud Pricing Calculator.” [Online]. Available: <https://cloud.google.com/products/calculator>
- [8] Microsoft Azure, “Azure Pricing Calculator.” [Online]. Available: <https://azure.microsoft.com/en-us/pricing/calculator/>
- [9] Datadog, “Cloudcraft: Visualize your cloud architecture.” [Online]. Available: <https://www.cloudcraft.co/>
- [10] xyflow, “React Flow: Customizable library for node-based editors and diagrams.” [Online]. Available: <https://reactflow.dev/>
- [11] “BD Tech Jobs: Specialized Software Engineering Portal.” Accessed: Jan. 22, 2026. [Online]. Available: <https://bdtechjobs.com/>
- [12] “TechnTalents: Automated ICT Job Platform Bangladesh.” Accessed: Jan. 22, 2026. [Online]. Available: <https://techntalents.com/>
- [13] “eJobs Bangladesh: Skill-Based IT Job Search.” Accessed: Jan. 22, 2026. [Online]. Available: <https://ejobs.com.bd/>
- [14] “JobMatchingBD: IT Careers and Internships in Dhaka.” Accessed: Jan. 22, 2026. [Online]. Available: <https://jobmatchingbd.com/>
- [15] “Arc.dev: Remote Senior Developer Roles in Bangladesh.” Accessed: Jan. 22, 2026. [Online]. Available: <https://arc.dev/>
- [16] “Wellfound: Startup and Full-Stack Roles in Bangladesh.” Accessed: Jan. 22, 2026. [Online]. Available: <https://wellfound.com/>
- [17] “Careerjet Bangladesh: Tech Job Search Engine.” Accessed: Jan. 22, 2026. [Online]. Available: <https://www.careerjet.com.bd/>

THE END